



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

Μάθημα: Ρομποτικά Συστήματα ΙΙ

3^η ατομική εργασία

Σάλιαρης Γεώργιος - ΑΜ: 1092584

**Υπεύθυνος καθηγητής: Κωνσταντίνος
Χατζηλυγερούδης**

Ακαδημαϊκό έτος: 2025-26

1o Task

Έχουμε το παρακάτω *double pendulum* σύστημα:

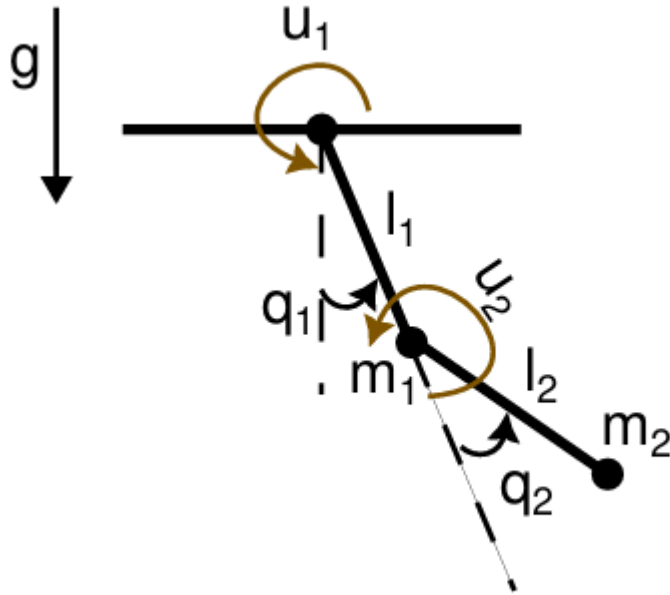


Figure 1: Double Pendulum System.

το οποίο περιγράφεται από τις εξισώσεις:

$$\mathbf{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^4 \quad (1.1)$$

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \in \mathbb{R}^2 \quad (1.2)$$

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} \in \mathbb{R}^4 \quad (1.3)$$

By defining $\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$, we can derive the following equation:

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{u} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}))$$

where

$$\begin{aligned} \mathbf{M} &= \begin{bmatrix} l_1^2 m_1 + l_2^2 m_2 + l_1^2 m_2 + 2l_1 m_2 l_2 \cos q_2 & l_2^2 m_2 + l_1 m_2 l_2 \cos q_2 \\ l_2^2 m_2 + l_1 m_2 l_2 \cos q_2 & l_2^2 m_2 \end{bmatrix} \in \mathbb{R}^{2 \times 2} \\ \mathbf{C} &= \begin{bmatrix} -2\dot{q}_2 l_1 m_2 l_2 \sin q_2 & -\dot{q}_2 l_1 m_2 l_2 \sin q_2 \\ \dot{q}_1 l_1 m_2 l_2 \sin q_2 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2} \\ \mathbf{G} &= \begin{bmatrix} -gm_1 l_1 \sin q_1 - gm_2 (l_1 \sin q_1 + l_2 \sin(q_1 + q_2)) \\ -gm_2 l_2 \sin(q_1 + q_2) \end{bmatrix} \in \mathbb{R}^{2 \times 1} \end{aligned}$$

and $g = 9.81 \text{ m/s}^2$, $m_1 = m_2 = 1 \text{ kg}$, $l_1 = l_2 = 0.5 \text{ m}$.

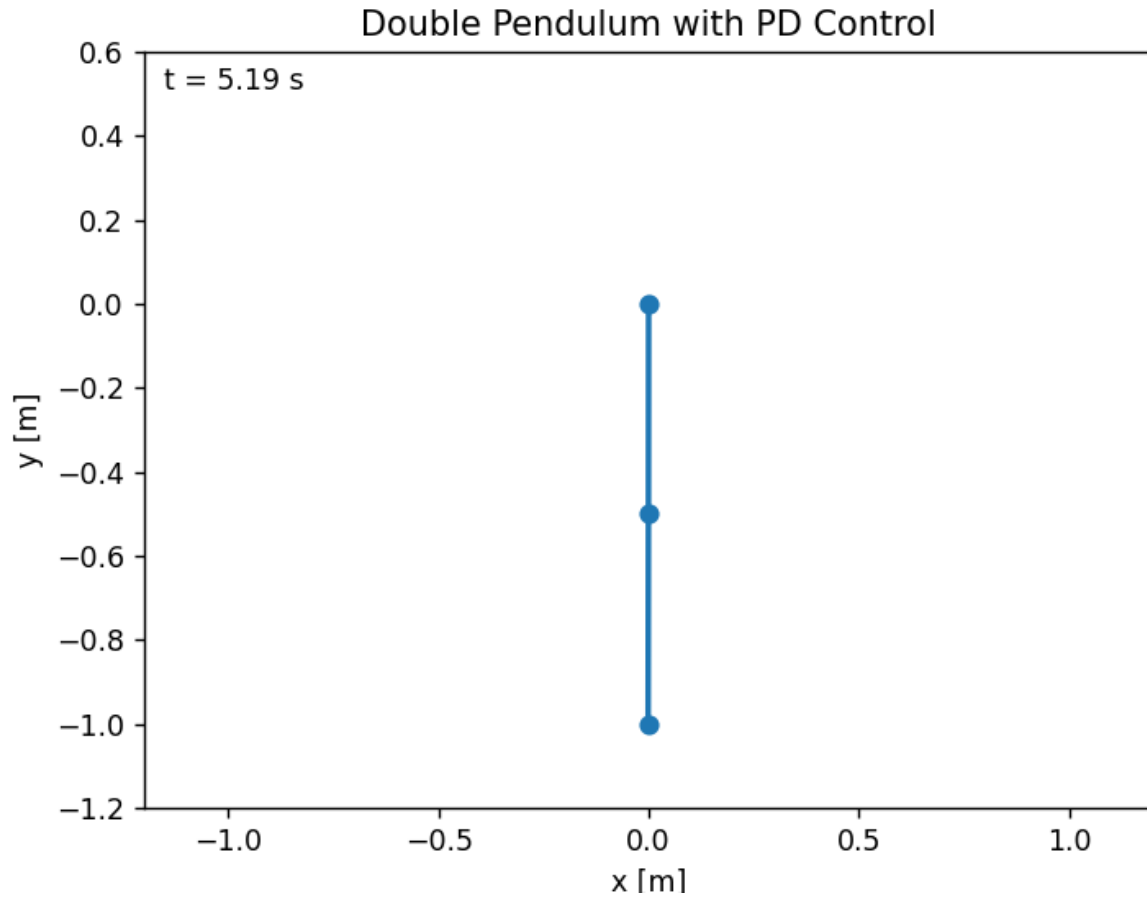
(1.4)

Και το προσομοιώνουμε το σύστημα μέσω *RK4*, χρησιμοποιώντας έναν *PD* ελεγκτή.

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) \\ t_{n+1} &= t_n + h \end{aligned} \tag{1.5}$$

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + h \frac{k_1}{2}\right), \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + h \frac{k_2}{2}\right), \\ k_4 &= f(t_n + h, y_n + h k_3). \end{aligned} \tag{1.6}$$

Τρέχοντας το αρχείο *RS_II_HW_3_1.py*, λαμβάνουμε τα εξής αποτελέσματα:



Εικόνα 1.2: Ξεκινώντας από την αρχική κατάσταση $x_0 = [0.6 \quad -0.3 \quad 0 \quad 0]^T$ (1.7), με τον *PD* ελεγκτή θέλει περίπου 5s για να ισορροπήσει στην κατάσταση ισορροπίας $x_{eq} = [0 \quad 0 \quad 0 \quad 0]^T$ (1.8).

Προκειμένου να επιβεβαιωθεί η ορθότητα της προσομοίωσης, επιλέγουμε ένα πολύ μικρό dt , και προσομοιώνουμε το σύστημά μας, οπότε προκύπτει ένα *reference trajectory* x_{ref} . Εν συνεχεία, υπολογίζουμε το *error* (για κάθε χρονική στιγμή t_k) στο διακριτοποιημένο σύστημα:

$$e_k = q(t_k) - q_{ref} \quad (1.9)$$

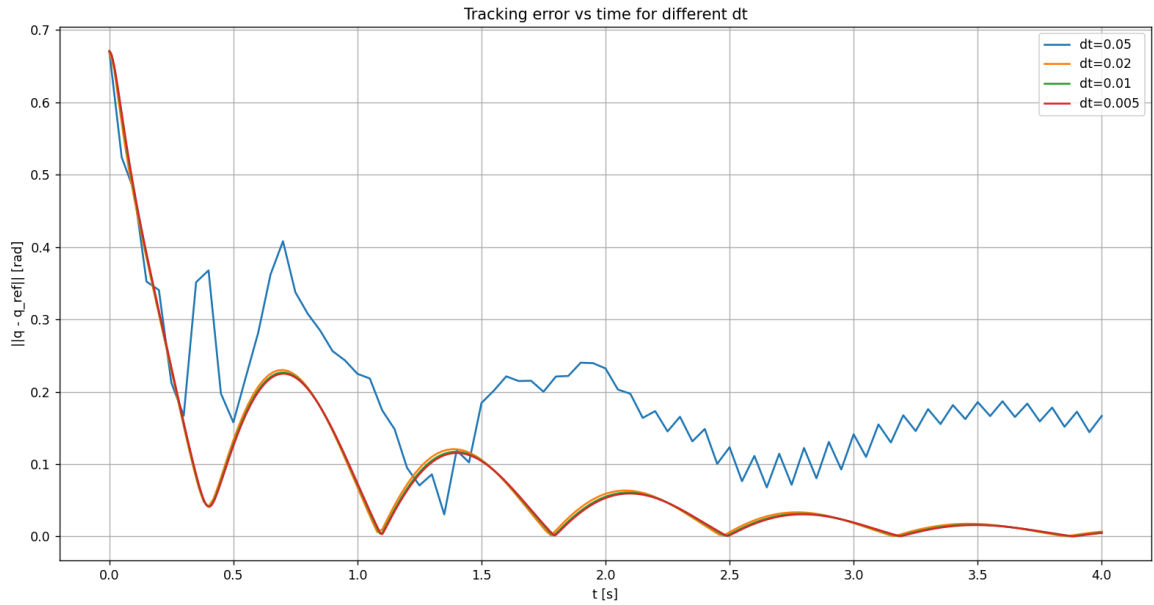
Και το RMS error:

$$RMS = \sqrt{\frac{1}{N+1} \sum_{k=0}^N ||e_k||_2^2} \quad (1.9),$$

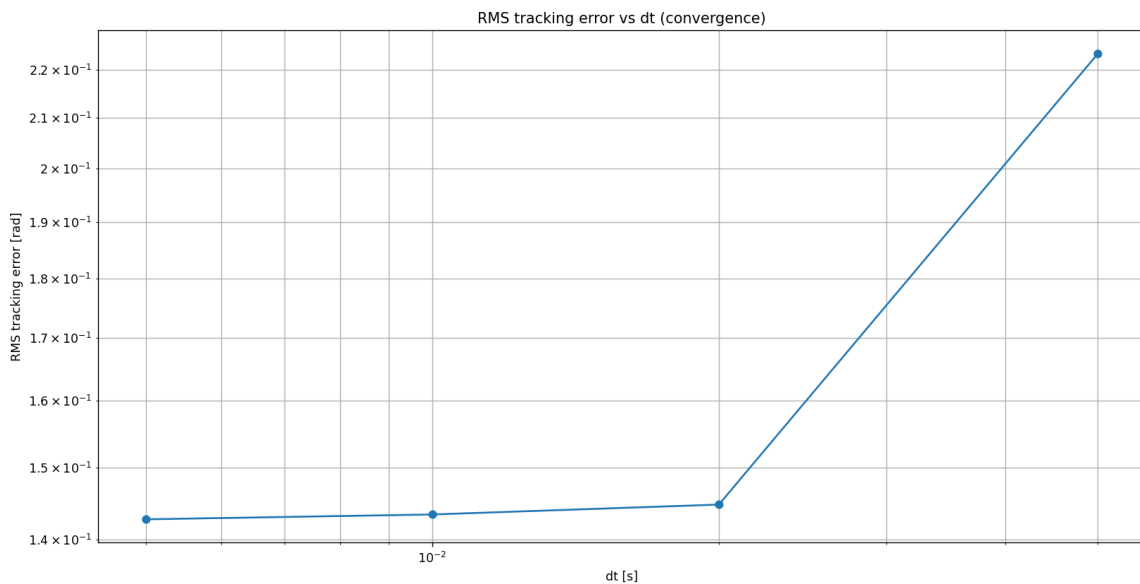
όπως επίσης και το *max state error*:

$$e_{max,state} = \max |x(t_k) - x_{ref}(t_k)|, \text{ για όλα τα } t_k \text{ (1.10).}$$

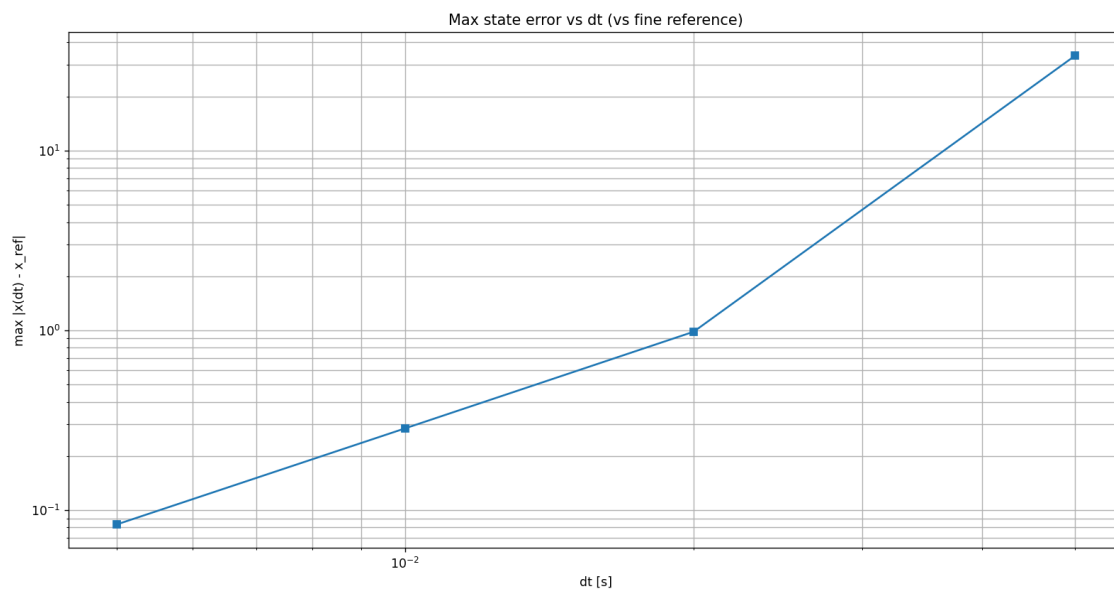
Οπότε και λαμβάνουμε τα παρακάτω γραφήματα:



Εικόνα 1.3: *Error vs χρόνο για διάφορα dt . Όσο μικραίνουμε τον χρόνο τόσο μικραίνει και το σφάλμα.*



Εικόνα 1.4: *RMS error vs dt . Πάλι για μικρότερα dt , μικραίνει και το error.*



Εικόνα 1.5: *Max state error vs dt*. Μεγαλώνει το dt , απομακρυνόμαστε από το reference, μεγαλώνει και αυτό το error.

2o Task

Τώρα έχουμε τους παρακάτω περιορισμούς:

- Initial state: $x_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
 - Final state: $x_f = \begin{bmatrix} \pi \\ 0 \\ 0 \\ 0 \end{bmatrix}$
 - Control inputs: $\begin{bmatrix} -10 \\ -10 \end{bmatrix} \leq u \leq \begin{bmatrix} 10 \\ 10 \end{bmatrix}$
 - Total time: 5 s
 - Number of knot points: 101
 - Timestep (dt): 0.05 s
- (2.1)

Και θέλουμε να διατυπώσουμε –υπό τους παραπάνω περιορισμούς ένα *non linear trajectory optimization problem* και να το λύσουμε. Έχουμε το παρακάτω πίνακα/διάνυσμα των μεταβλητών απόφασης:

$$z = [x_0 \quad x_1 \quad \dots \quad x_{N-1} \quad u_0 \quad u_1 \quad \dots \quad u_{N-1}] \quad (2.2), \text{ με}$$

$$x_{k+1} = x_k + \frac{h}{6}(f_k + 4f_c + f_{k+1}) \quad (2.3)$$

$$f_k = f(x_k, u_k) \quad (2.4)$$

$$f_{k+1} = f(x_{k+1}, u_{k+1}) \quad (2.5)$$

$$x_{mid} = \frac{(x_k + x_{k+1})}{2} + \frac{h}{8}(f_k - f_{k+1}) \quad (2.6)$$

$$u_{mid} = \frac{u_k + u_{k+1}}{2} \quad (2.7)$$

$$f_{mid} = f(x_{mid}, u_{mid}) \quad (2.8)$$

$$x_{k+1} = x_k + \frac{h}{6}(f_k + 4f_{mid} + f_{k+1}) \quad (2.9)$$

Τις εξισώσεις (2.1) – (2.9), τις κάνουμε *stack* σε ένα διάνυσμα της μορφής $g(z) = 0$ (2.10), οπότε έχουμε να βελτιστοποιήσουμε τη συνάρτηση

$$\min_{x_0, \dots, x_{N-1}, u_0, \dots, u_{N-1}} \sum_{k=0}^{N-2} \frac{h}{2} (u_k^T R u_k + u_{k+1}^T R u_{k+1}) + w_{\Delta u} \sum_{k=0}^{N-2} \|u_{k+1} - u_k\|^2 \quad (2.11),$$

υπό τις (2.1) και (2.11).

Η λογική της παραπάνω εξίσωσης είναι ότι θέλουμε να βρούμε ροπές που οδηγούν το σύστημα από την αρχική στην τελική κατάσταση, αλλά χωρίς υπερβολικό ή άγαρμπο *effort*. Γι' αυτό το *cost function* που ορίζουμε έχει δύο βασικές ιδέες: πρώτον, τιμωρούμε τις μεγάλες ροπές, ώστε να αποφεύγονται λύσεις που βασίζονται σε ακραία *control inputs* και θα ήταν δύσκολο ή μη ρεαλιστικό να εφαρμοστούν σε πραγματικούς κινητήρες. Δεύτερον, τιμωρούμε τις απότομες αλλαγές της ροπής από τη μία χρονική στιγμή στην επόμενη, ώστε το σήμα ελέγχου να μη “κομπιάζει” και να μη πηδάει συνεχώς πάνω–κάτω, κάτι που κάνει την κίνηση “νευρική” και δύσκολη στην υλοποίηση. Επομένως, βρίσκουμε λύσεις που πετυχαίνουν τον στόχο με ροπές όσο γίνεται πιο οικονομικές (ενεργειακά) και ταυτόχρονα όσο γίνεται πιο ομαλές και φυσικές. Το βάρος $w_{\Delta u}$ ρυθμίζει το πόσο θέλουμε να έχουμε μεγάλες αλλαγές στα torques οπότε και να έχουμε απότομες αλλαγές στην κίνηση του συστήματος.

Έχοντας πάρει τα αποτελέσματα από τον *optimizer*, θεωρούμε:

$$x(s) = h_{00}(s) x_k + h_{10}(s) (h f_k) + h_{01}(s) x_{k+1} + h_{11}(s) (h f_{k+1}) \quad (2.12)$$

Με

$$s = \frac{t - t_k}{h} \quad (2.13)$$

$$\begin{aligned} h_{00}(s) &= 2s^3 - 3s^2 + 1 \\ h_{10}(s) &= s^3 - 2s^2 + s \\ h_{01}(s) &= -2s^3 + 3s^2 \\ h_{11}(s) &= s^3 - s^2 \end{aligned} \quad (2.14)$$

Οπότε, στο διάστημα $[t_k, t_{k+1}]$, έχουμε ένα σχετικά *smooth state*.

Για το δε έλεγχο:

$$u(t) = (1 - s) u_k + s u_{k+1}$$

(2.15)

Τρέχοντας το αρχείο *RS_II_HW_3_2.py* λαμβάνουμε τα εξής αποτελέσματα:

Figure 1: Double pendulum angles (smooth cubic Hermite reconstruction)

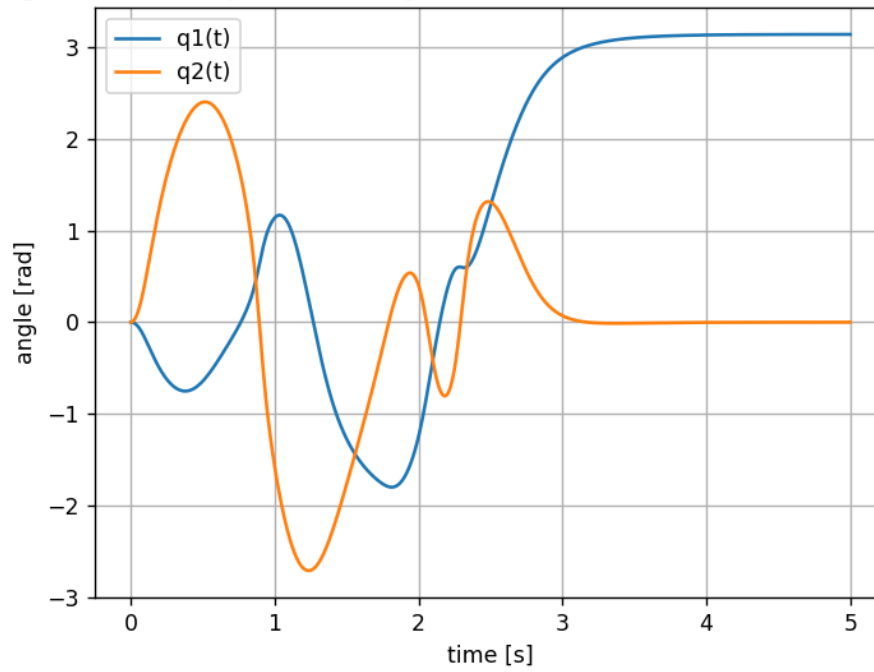


Figure 2: Double pendulum velocities (smooth cubic Hermite reconstruction)

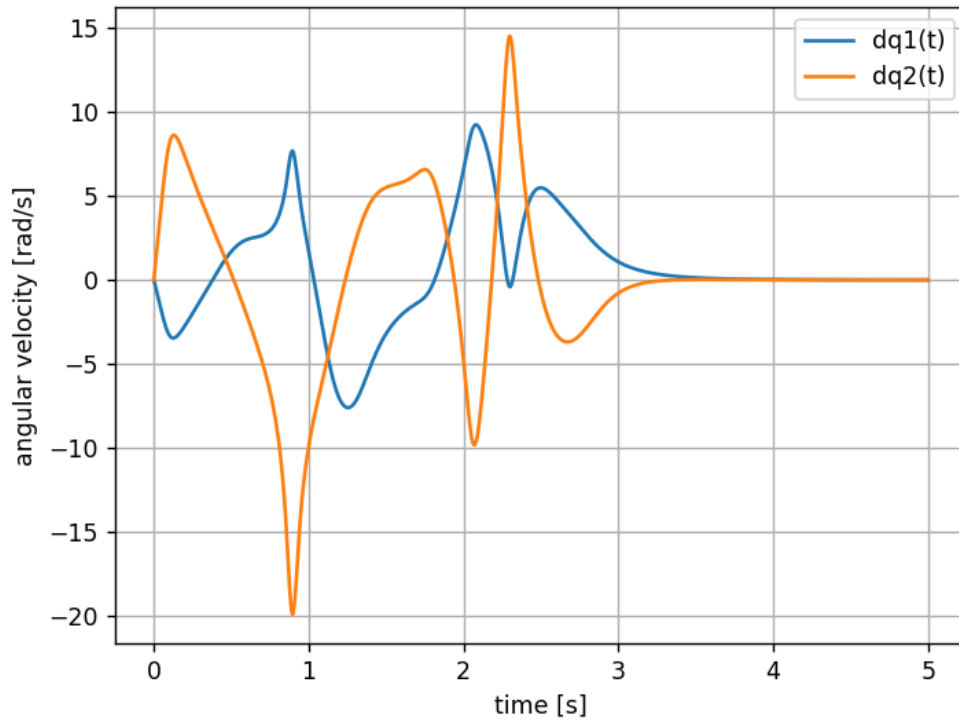
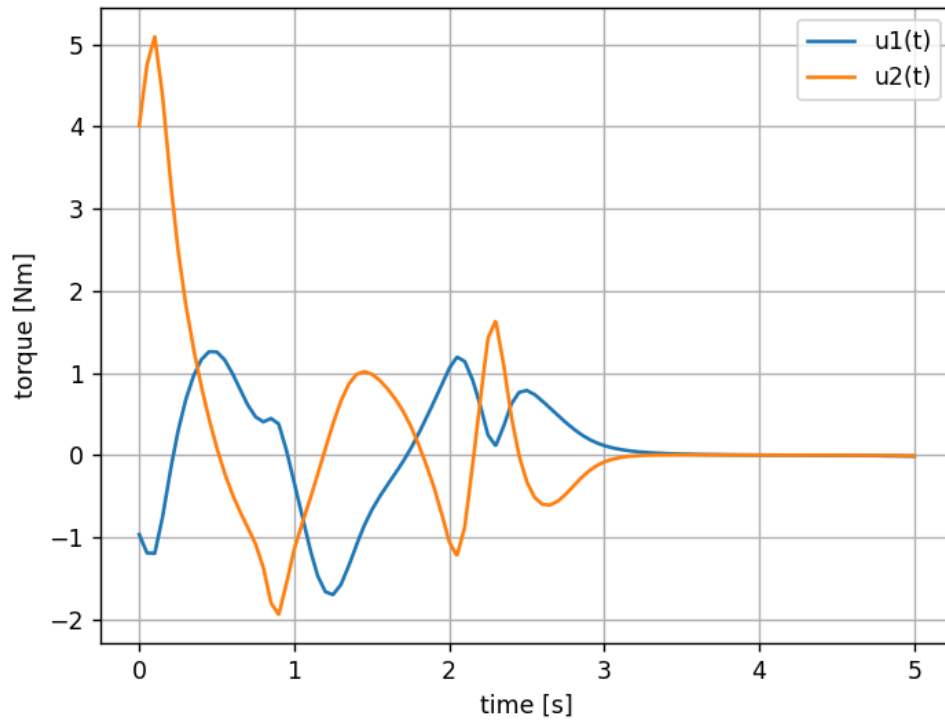
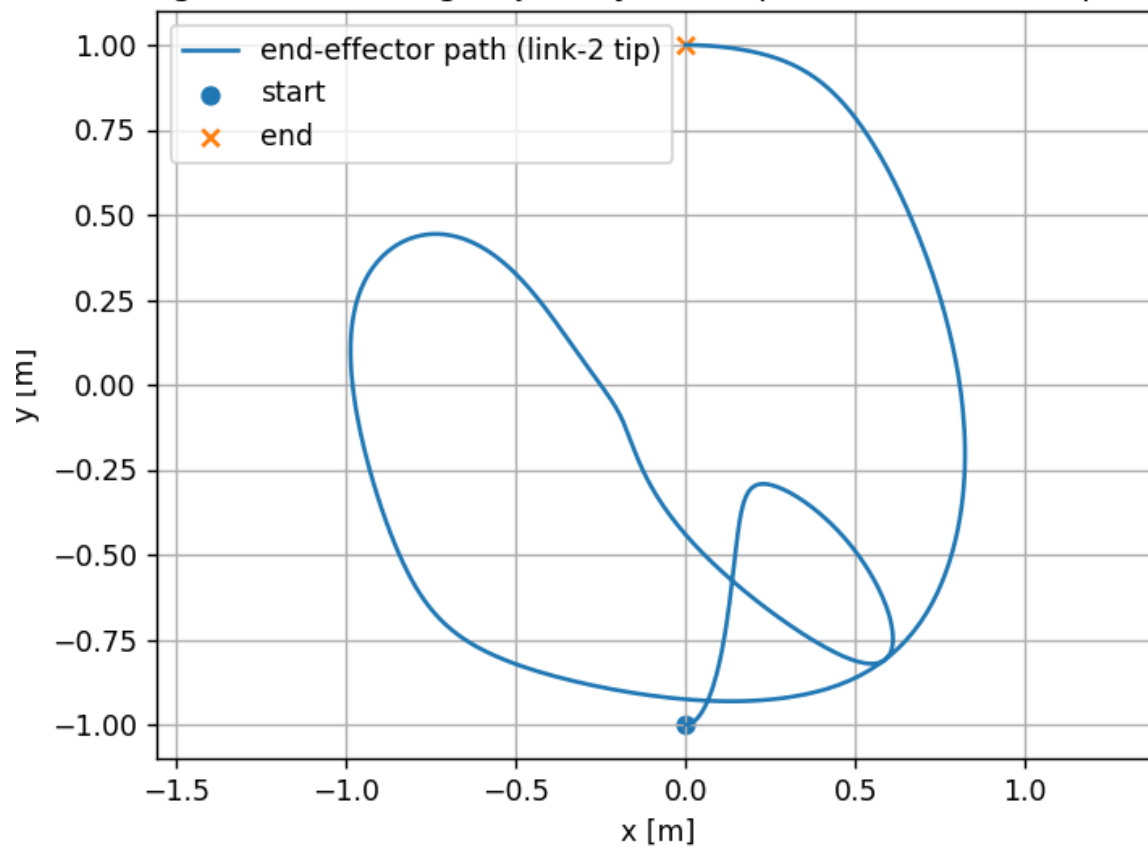


Figure 3: Control inputs (piecewise linear interpolation)



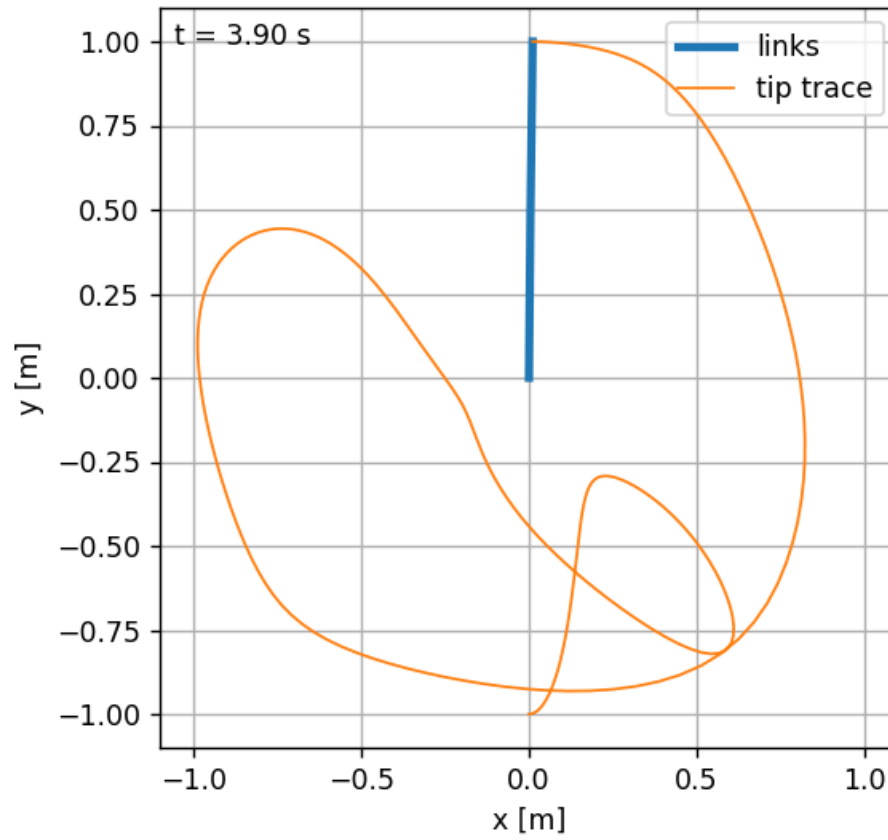
Εικόνες 2.1, 2.2, 2.3: Γωνίες,γωνιακές ταχύτητες και control inputs του double-pendulum. Παρατηρούμε ότι επέρχεται ισορροπία.

Figure 4: Resulting trajectory in XY space (end-effector path)



Εικόνα 2.4: Trajectory που ακολουθεί η μία άκρη του *pendulum*.

Figure 5: Live animation of the double pendulum (optimal trajectory)



Εικόνα 2.5: Στιγμιότυπο από το live animation της optimal τροχιάς. Σε περίπου 4s έχουμε ισορροπία.

3o Task

Αποτελεί συνέχεια του 2^{ου} task. Εμείς έχοντας λύσει το πρόβλημα του *trajectory optimization*, έχουμε βρει μια ακολουθία $\{\bar{x}_k, \bar{u}_k\}_{k=0}^{N-1}$ και θέλουμε να σχεδιάσουμε έναν *TVLQR controller*.

Για κάθε ζεύγος $(\bar{x}_k, \bar{u}_k)$ γραμμικοποιούμε το σύστημά μας, δηλαδή:

$$\Delta x_{k+1} = A_k \delta x_k + B_k \delta u_k \quad (3.1)$$

$$\delta x_k = x_k - \bar{x}_k, \quad \delta u_k = u_k - \bar{u}_k \quad (3.2)$$

(έχουμε προσθέσει γκαουσιανό θόρυβο στα x_k).

$$A_k = \frac{\partial f_{disc}}{\partial x} \big|_{(\bar{x}_k, \bar{u}_k)}, B_k = \frac{\partial f_{disc}}{\partial u} \big|_{(\bar{x}_k, \bar{u}_k)} \quad (3.3)$$

Το δε *cost function* θα είναι:

$$J = \sum_{k=0}^{N-2} (\delta x_k^T Q \delta x_k + \delta u_k^T R \delta u_k) + \delta x_{N-1}^T Q_f \delta x_{N-1} \quad (3.4)$$

Εμείς κάνουμε *Riccati recursion*, οπότε για κάθε χρονική στιγμή k , έχουμε και τον ελεγκτή (αυτή είναι και η διαφορά με το απλό *LQR*, ότι έχουμε διαφορετικό ελεγκτή για κάθε χρονική στιγμή k):

$$u_k = \bar{u}_k - K_k(x_k - \bar{x}_k) \quad (3.5)$$

Έχουμε όμως το πρόβλημα ότι μπορεί να θέλουμε να τρέξουμε το *simulation* σε διαφορετικό *timestep* από το *nominal trajectory*, οπότε θα πρέπει να είμαστε σε θέση να υπολογίζουμε τον πίνακα K (για τα $\bar{x}_k, \bar{u}_k$ θεωρούμε *ZOH* ή μπορούμε να εφαρμόσουμε την ίδια λογική με που εφαρμόζουμε και για τον πίνακα K). Εφαρμόζουμε *Kernel regression*:

$$K(t) = \frac{\sum_i w_i(t) K_i}{\sum_i w_i(t)} \quad (3.6)$$

με

$$w_i(t) = \exp\left(-\frac{1}{2} \left(\frac{t - t_i}{h}\right)^2\right) \quad (3.7)$$

Εφαρμόζουμε και τεχνική projection στον ελεγκτή μας. Ειδικότερα υπολογίζουμε ($\hat{x}$ = η πραγματική κατάσταση του συστήματος + θόρυβος, επιλέγοντας έναν θετικά ορισμένο πίνακα W).

Υπολογίζουμε την παρακάτω ποσότητα για κάθε *knot point*:

$$d_k^2 = (\hat{x} - \bar{x}_k)^T W (\hat{x} - \bar{x}_k) \quad (3.8)$$

Σχηματίζουμε το σύνολο:

$$A = \{k | d_k \leq r\} \quad (3.9)$$

Εαν δεν είναι κενό:

$$k_{ref} = \max(A) \quad (3.10)$$

Εάν είναι κενό:

$$k_{ref} = \operatorname{argmin}_k d_k \quad (3.11)$$

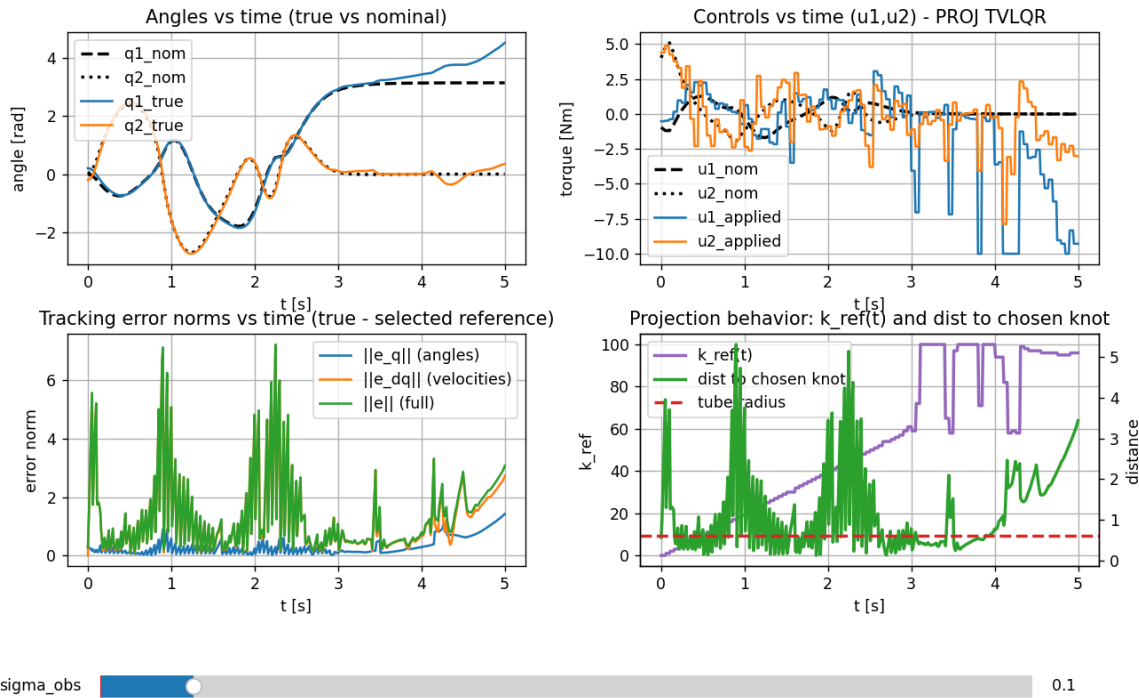
Οπότε έχουμε την χρονική στιγμή $t_{ref} = t_{k_{ref}}$

Οπότε εν τέλει ο ελεγκτής μας θα είναι $K(t_{ref}) \rightarrow$ ο πίνακας K υπολογισμένος την t_{ref} :

$$u = \bar{u}_{k_{ref}} - K(t_{ref})(\hat{x} - \bar{x}_{k_{ref}})$$

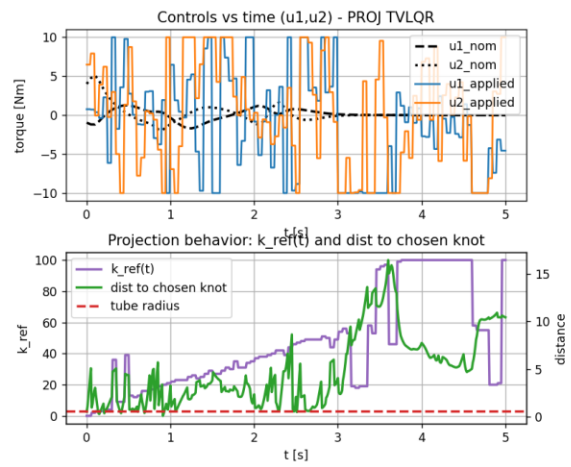
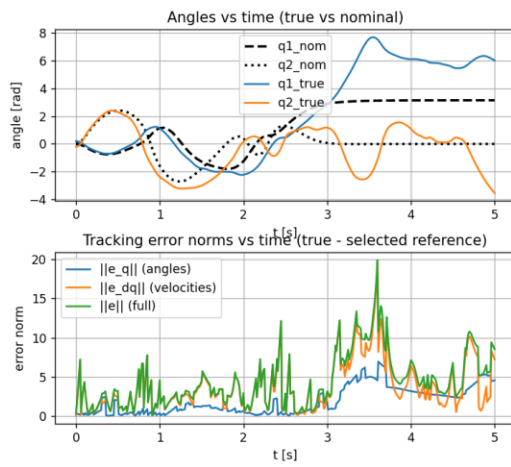
Αυξάνοντας το θόρυβο, παρατηρούμε ότι το σύστημα απομακρύνεται ολοένα και περισσότερο από την τελική επιθυμητή θέση ισορροπίας, αλλά δεν σπάει ο ελεγκτής, δεν κάνει δηλαδή το σύστημά μας περίεργες κινήσεις, πράγμα που αποδεικνύει το *robustness* του ελεγκτή.

Τρέχοντας το αρχείο *RS_II_HW_3.py*, λαμβάνουμε τα ακόλουθα γραφήματα:



Εικόνα 3.1: Αυτή η εικόνα δείχνει συνολικά αν ο *PROJ-TV LQR* “κρατάει” το σύστημά μας πάνω στη *nominal* τροχιά. Πάνω αριστερά βλέπουμε τις γωνίες: οι μαύρες είναι η *nominal*, οι χρωματιστές είναι το πραγματικό σύστημα. Πάνω δεξιά βλέπουμε τις ροπές: οι μαύρες είναι οι *nominal* ροπές, ενώ οι δύο **συνεχόμενες γραμμές** είναι οι εφαρμοσμένες—**μπλε = $u1_applied$** , **πορτοκαλί = $u2_applied$** . Κάτω αριστερά βλέπουμε είναι πόσο μεγάλο είναι το σφάλμα (γωνίες/ταχύτητες/σύνολο) και κάτω δεξιά φαίνεται τι κάνει το *projection*: ποιο *knot* διαλέγεται σαν αναφορά (k_ref) και πόσο μακριά είμαστε από αυτό. Αν το k_ref πηδάει απότομα ή η απόσταση είναι συχνά μεγάλη, ενδεχομένως να έχεις χαλάσει και το *tracking*. Παρατηρούμε καλό *tracking*. Αυξάνοντας τον θόρυβο, χαλάει το *tracking*, όπως φαίνεται και από την παρακάτω εικόνα.

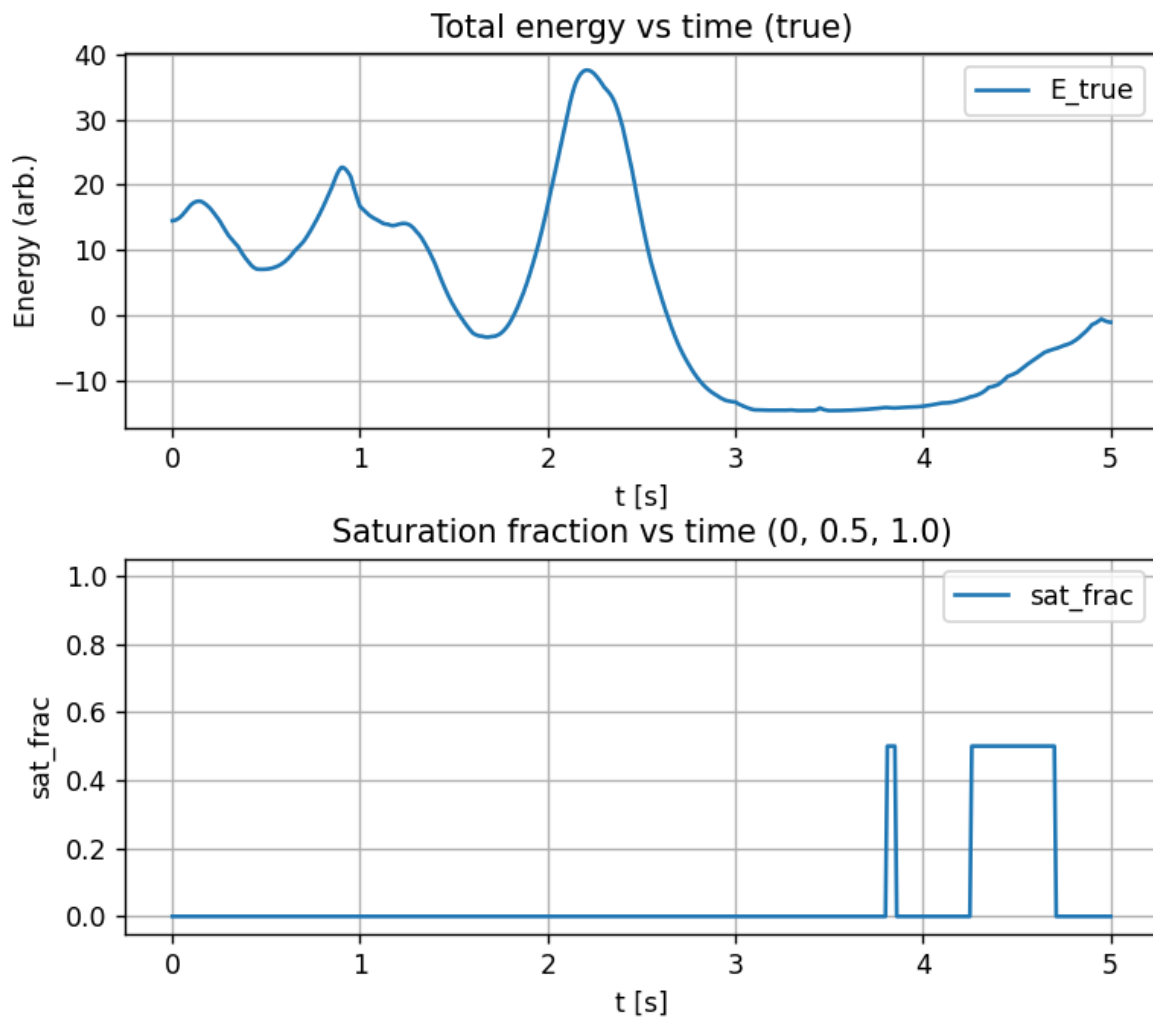
Εικόνα 3.2: Αυξάνοντας τον θόρυβο, έχουμε λιγότερο καλό *tracking*



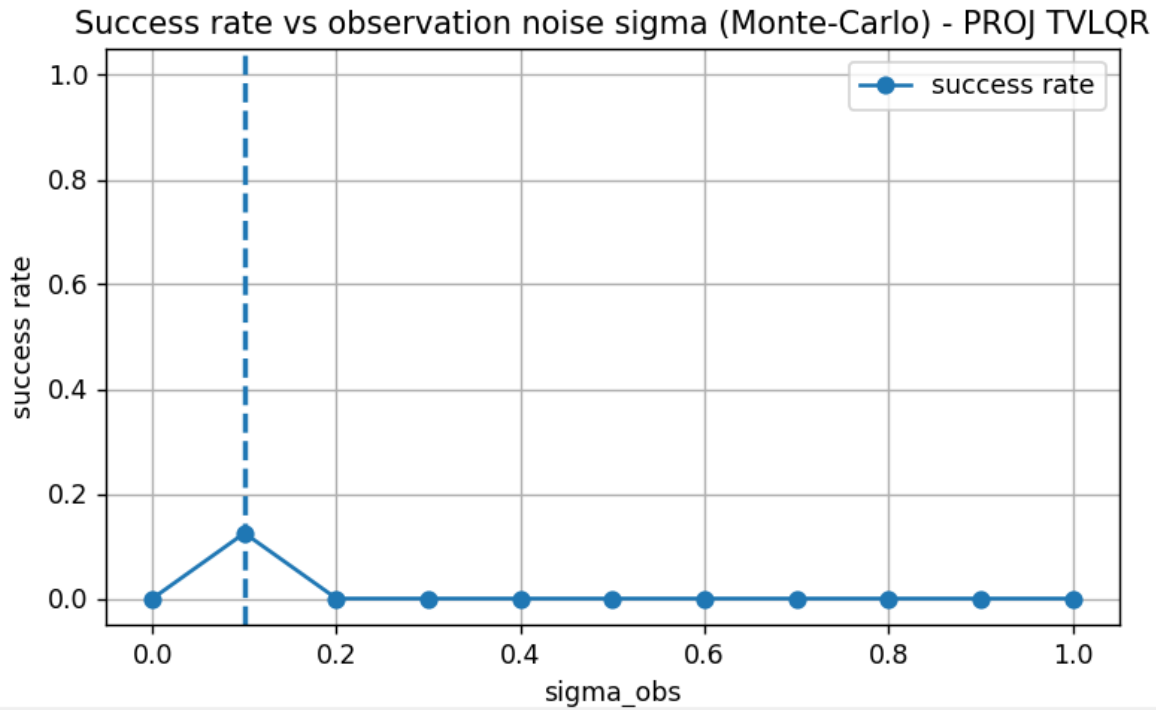
sigma_obs



1

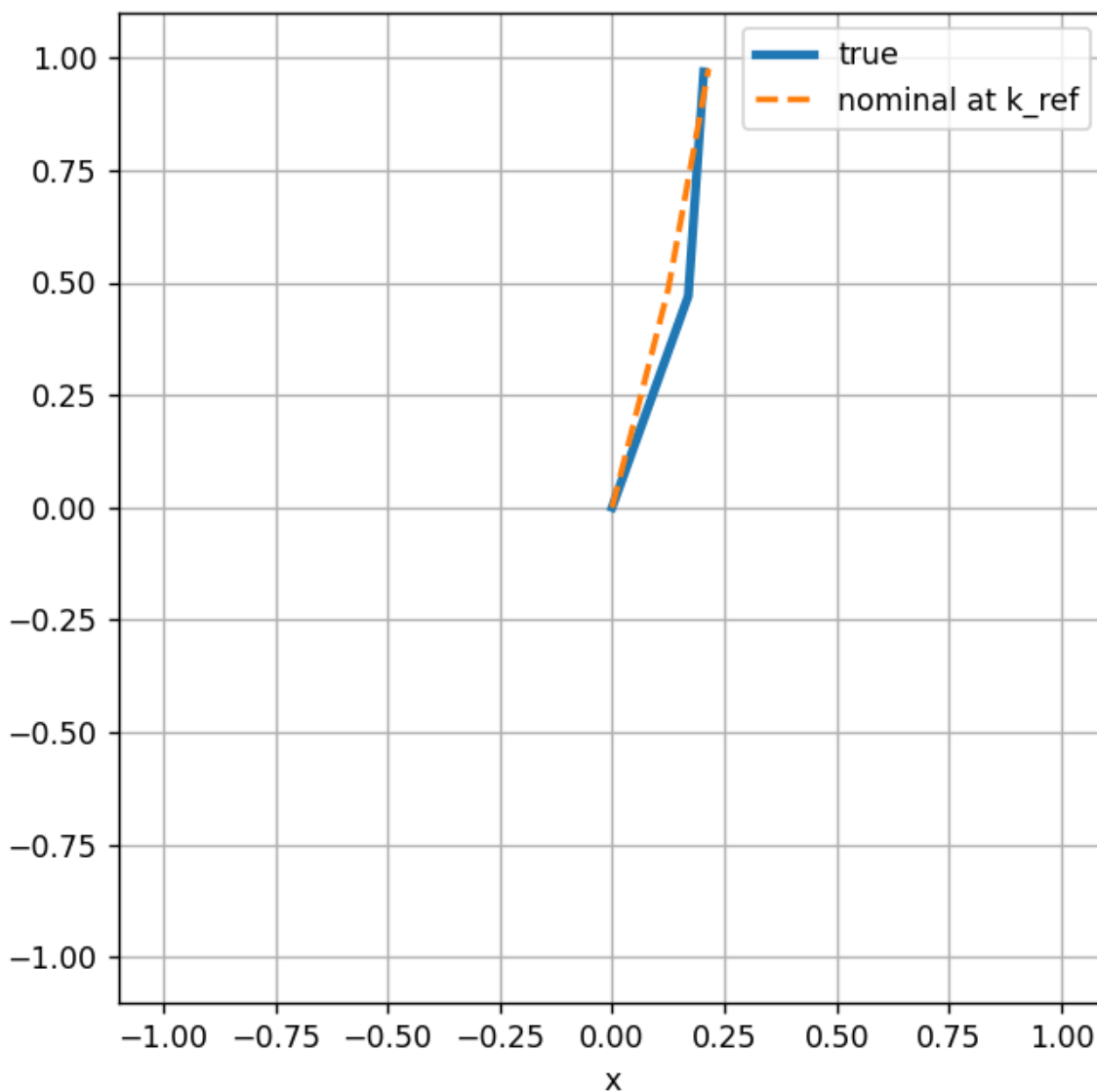


Εικόνα 3.3 Εδώ έχω δύο διαγνωστικά που εξηγούν γιατί μπορεί να αποτυγχάνει το *tracking*: στο πάνω μέρος η **ολική ενέργεια** δείχνει πόσο “φορτώνεται” ή “εκτονώνεται” το σύστημα στο χρόνο, και απότομα ανεβοκατεβάσματα συνήθως σημαίνουν ότι ο *controller* βάζει/βγάζει ενέργεια με έντονο τρόπο (συχνά επειδή προσπαθεί να διορθώσει μεγάλα σφάλματα). Στο δε κάτω μέρος, το **saturation fraction** δείχνει πόσο συχνά οι εντολές ροπής χτυπάνε τα όρια: 0 σημαίνει κανένα *torque* δεν κάνει *saturate*, 0.5 σημαίνει ότι κάνει *saturate* περίπου το ένα από τα δύο ($u1$ ή $u2$), και 1.0 σημαίνει ότι κάνουν *saturate* και τα δύο. Αρα τα μεγάλα διαστήματα στο 0.5 ή 1.0, είναι ένδειξη ότι το σύστημα δουλεύει “εκτός υποθέσεων *LQR*” και αυτό συνήθως εξηγεί γιατί η πραγματική τροχιά δεν ακολουθεί τη *nominal* (ο *controller* ζητάει κάτι που κόβεται από το *clip* που κάνουμε λόγω των περιορισμών που έχουν τεθεί).



Εικόνα 3.4: Το γράφημα δείχνει πόσο συχνά “πετυχαίνει” ο ελεγκτής όταν αυξάνεται ο θόρυβος στη μέτρηση. Για κάθε επίπεδο θορύβου γίνονται πολλά runs και μετράμε το ποσοστό επιτυχίας. Φαίνεται ότι ακόμα και με μικρή αύξηση του θορύβου το ποσοστό πέφτει σχεδόν στο μηδέν, άρα ο *controller* είναι ιδιαίτερα ευαίσθητος σε θορυβώδεις παρατηρήσεις. Η διακεκομμένη κάθετη γραμμή δείχνει την τρέχουσα τιμή που έχουμε επιλέξει από το *slider*.

Live animation (true vs nominal at k_{ref})



Εικόνα 3.5: Στιγμιότυπο του *Live animation*. Η μπλε στάση είναι το πραγματικό σύστημα, και η πορτοκαλί διακεκομμένη είναι η *nominal* στάση στο *knot* που διάλεξε το *projection* (k_{ref}). Αν οι δύο στάσεις είναι κοντά, είμαστε οκ. Αν είναι μακριά, ο *controller* κινηγεί λανθασμένο σημείο της τροχιάς το οποίο έχει ως αποτέλεσμα μεγάλα σφάλματα και ροπές.